

# 基于滑动窗口线性回归的 IoTDB 短期趋势预测 UDF 选题报告

## 摘要

我们面向 IoTDB 中常见的传感器时间序列数据，计划实现一个基于滑动窗口线性回归的短期趋势预测 UDF。该算子以数据库中的数值型时间序列为输入，在查询过程中维护一个固定长度的滑动窗口，对窗口内数据进行局部线性拟合，并输出短期预测值或局部趋势信息。

我们采用的线性回归算法本身较为基础，但它具有计算开销低、参数直观、结果可解释等特点。将其封装为 IoTDB UDF 后，可以使用户在数据库查询阶段直接完成轻量级趋势分析，减少数据导出和外部脚本处理的成本。因此，我们的重点不只是算法公式本身，而是如何将该方法设计成适合时序数据库场景的流式 UDF 算子，并在效率、窗口维护、参数控制和实验验证方面形成较完整的实现方案。

## 一、研究背景与问题定义

### 1.1 应用场景

Apache IoTDB 主要用于时序数据的存储和查询，适用于工业设备监测、能源系统、环境传感器、通信基站监控等场景。这类场景中的数据通常会按照时间连续产生，例如设备温度、电机振动、电网负荷、水位流量、空气质量指标等。

在实际应用中，用户除了查询历史数据，还经常需要判断某个指标的短期变化趋势。例如，设备温度是否正在持续升高，负载是否可能接近阈值，某个传感器指标是否出现持续下降趋势等。如果每次都从 IoTDB 中导出数据，再使用外部程序进行分析，会增加数据搬运和脚本维护成本。因此，将轻量级趋势预测能力封装为 IoTDB UDF，具有一定的实际意义。

### 1.2 领域数据特点

IoTDB 中的物联网时间序列通常具有以下特点：

- 时间顺序明显**：数据以时间戳为核心组织，预测结果依赖最近一段历史数据的顺序。
- 数据规模较大**：传感器可能持续高频写入，算子不适合保存完整历史序列。
- 局部趋势较重要**：长期数据未必满足线性关系，但短时间窗口内常可以近似看作上升、下降或平稳。
- 存在噪声和波动**：单个采样点可能受测量误差影响，因此需要从一段局部数据中提取整体趋势。
- 结果需要可解释**：在监测场景中，斜率、预测值等结果比复杂模型更容易理解和展示。

### 1.3 研究动机

我们希望实现一个简单但实用的 IoTDB UDF 算子。滑动窗口线性回归不是复杂算法，但它与时序数据库的查询机制结合后，可以直接服务于数据库内的趋势分析。

相比移动平均，线性回归不仅能起到一定的平滑作用，还能给出趋势方向和短期外推结果；相比复杂时序模型，它不需要训练过程，也不依赖外部模型服务，因而无论是实现上还是在实际使用中成本都相对低廉。因此，该方法适合作为预测类 UDF 的基础实现对象。

## 1.4 问题目标

给定 IoTDB 中某条数值型时间序列，目标是在查询过程中按时间顺序处理数据，并对每个局部窗口进行短期趋势估计。具体目标包括：

- 1. 支持对单列数值型时间序列进行滑动窗口处理；
- 2. 在窗口内完成线性回归拟合；
- 3. 根据拟合结果输出短期预测值或趋势斜率；
- 4. 支持窗口长度、预测步长等参数配置；
- 5. 在实现中兼顾计算效率、内存占用和边界情况处理。

| 输入        | 输出           | 核心参数            | 预期作用                  |
|-----------|--------------|-----------------|-----------------------|
| 一列数值型时间序列 | 短期预测值或局部趋势信息 | 窗口长度、预测步长、输出模式等 | 在 IoTDB 查询阶段完成轻量级趋势预测 |

## 1.5 相关方法对比

针对时间序列短期预测，常见方法包括移动平均、指数平滑、ARIMA、机器学习模型以及深度学习模型等。不同方法适用的场景和实现成本不同。我们选择滑动窗口线性回归，主要是考虑到课程作业中需要完成一个可落地的 UDF 算子，而不是训练和部署复杂模型。

| 方法          | 特点         | 优点             | 局限                        |
|-------------|------------|----------------|---------------------------|
| 移动平均        | 对最近若干点求平均  | 实现简单，能平滑噪声     | 主要反映均值，不能直接给出趋势斜率         |
| 指数平滑        | 对新数据赋予更高权重 | 对短期变化更敏感       | 参数选择会影响结果，解释性略弱于线性斜率      |
| ARIMA 等统计模型 | 建模自相关结构    | 对某些平稳序列表现较好    | 参数识别和模型假设较复杂，不适合简单 UDF 起步 |
| 机器学习/深度学习模型 | 可表达复杂非线性关系 | 预测能力上限较高       | 需要训练、模型管理和更多计算资源。时延大多较大。  |
| 滑动窗口线性回归    | 在局部窗口内拟合直线 | 简单、可解释、易封装、开销低 | 只能刻画局部近似线性趋势              |

因此，我们并不是为了追求最复杂的预测模型，而是希望实现一个低成本、可解释、与 IoTDB 查询流程结合紧密的基础预测算子。

# 二、算法介绍与设计思路

## 2.1 核心算法

我们采用普通最小二乘线性回归。对于当前窗口内的  $n$  个数据点，设第  $i$  个点的横坐标为  $x_i$ ，纵坐标为  $y_i$ 。在等间隔采样场景下，可以将窗口内数据点的序号作为横坐标，即：

$$x_i = i, \quad i = 0, 1, \dots, n-1 \quad (1)$$

线性回归拟合的目标是得到一条直线：

$$y = a + bx \quad (2)$$

其中， $a$  表示截距， $b$  表示斜率。斜率  $b$  可以反映当前窗口内的局部趋势：当  $b > 0$  时表示整体上升，当  $b < 0$  时表示整体下降，当  $b$  接近 0 时表示短期内较平稳。

普通最小二乘法通过最小化窗口内所有点到拟合直线的平方误差来求解参数：

$$\min_{a,b} \sum_{i=0}^{n-1} (y_i - a - bx_i)^2 \quad (3)$$

对应的斜率和截距可由下式计算：

$$b = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} \quad (4)$$

$$a = \bar{y} - b\bar{x} \quad (5)$$

得到拟合直线后，可以对未来第  $h$  步进行外推预测：

$$\hat{y} = a + b(n-1+h) \quad (6)$$

其中， $h$  表示预测步长， $\hat{y}$  表示预测值。

## 2.2 滑动窗口思想

如果使用全部历史数据进行回归，较早的数据会对当前趋势造成干扰，不适合短期预测。因此，我们采用滑动窗口机制，只使用最近一段数据进行局部拟合。

滑动窗口的优势在于，它可以让算子更关注当前阶段的数据状态。当新数据到来时，窗口向前移动，旧数据被移出，新数据被加入，预测结果也随之更新。这样既避免了保存完整历史数据，也使算子更适合 IoTDB 中按时间顺序流式处理数据的方式。

窗口长度会影响预测效果。较小窗口对变化更敏感，但更容易受噪声影响；较大窗口结果更平滑，但对突发趋势变化反应较慢。因此，窗口长度需要作为可配置参数，并在实验中对不同参数进行比较。

## 2.3 设计思路

我们的整体设计思路是将“局部线性拟合”封装为数据库内的流式计算过程。算子不需要一次性加载整条时间序列，而是在查询过程中按行接收数据，维护当前窗口状态，并基于窗口内的统计量计算预测结果。

为了提高效率，基础实现可以直接遍历窗口计算回归参数；进一步优化时，可以维护窗口内的累加统计量，例如：

$$\sum x_i, \quad \sum y_i, \quad \sum x_i y_i, \quad \sum x_i^2 \quad (7)$$

当窗口滑动时，只更新进入窗口和离开窗口的数据对统计量的影响，从而减少重复计算。这样可以使算子在大规模时间序列查询中保持较低的计算开销。

## 2.4 参数设计

| 参数     | 含义               | 设计目的            |
|--------|------------------|-----------------|
| 窗口长度   | 控制每次参与拟合的数据点数量   | 平衡噪声抑制能力和趋势响应速度 |
| 预测步长   | 控制向未来外推的距离       | 支持一步预测或短期多步预测   |
| 最小有效点数 | 控制窗口未满时是否输出结果    | 避免数据点过少导致结果不稳定  |
| 横坐标模式  | 选择按点序号或按真实时间间隔建模 | 兼顾等间隔和非等间隔采样数据  |
| 输出模式   | 选择输出预测值或趋势斜率     | 支持数值预测和趋势判断两类需求 |

2.5 复杂度分析

在直接遍历窗口的基础实现中，假设窗口长度为  $w$ ，每处理一个新点需要遍历当前窗口，因此单点时间复杂度为：

$$O(w) \tag{8}$$

空间复杂度为：

$$O(w) \tag{9}$$

因为算子只需要保存当前窗口内的数据，而不需要保存完整历史序列。

如果采用增量统计量维护方式，在窗口滑动时只更新少量累加值，理论上可以将单点更新开销降低到接近：

$$O(1) \tag{10}$$

因此，我们在实现时可以先完成直接版本，保证逻辑清晰；在此基础上，再考虑通过增量统计优化运行效率。

2.6 适用范围与局限性

滑动窗口线性回归适合用于短期趋势相对平滑、局部变化近似线性的时序数据。例如设备温度缓慢升高、负载逐渐增加、环境指标短时间内缓慢波动等场景，都可以通过局部斜率和预测值获得较直观的结果。

但该方法也存在明显局限。首先，它无法表达复杂周期性和非线性变化；其次，窗口内如果出现极端异常值，拟合直线可能会被拉偏；最后，如果预测步长过长，线性外推的误差会迅速增加。因此，我们所说的“短期预测”应理解为基于最近窗口的局部外推，而不是对长期趋势进行准确预测。

三、系统实现方案

3.1 整体架构

我们拟将算法封装为 IoTDB 中的用户自定义表函数。系统整体上可以分为输入校验、窗口管理、趋势计算、结果输出和实验验证几个部分。

| 模块      | 主要职责                        |
|---------|-----------------------------|
| 输入与参数校验 | 检查输入序列类型、参数范围和输出模式是否合理      |
| 滑动窗口管理  | 维护当前窗口中的时间戳和值，控制旧数据移出和新数据进入 |
| 回归统计计算  | 根据窗口内数据计算斜率、截距和预测值          |
| 输出组织    | 将预测值或趋势信息以 IoTDB 支持的数据类型输出  |
| 实验验证    | 从正确性、趋势识别、抗噪声和性能等方面验证算子效果   |

这种结构可以使算法逻辑和 IoTDB 接口逻辑相对分离，后续如果需要替换预测方法或增加输出类型，也比较容易扩展。

### 3.2 流式处理方案

IoTDB UDF 在查询过程中通常按时间顺序处理数据，因此我们采用流式处理思路。算子每接收一个新的时间点，就更新当前窗口，并基于最新窗口计算结果。

这种方式有两个优点：

- **内存占用可控**：无论原始序列多长，算子只保存一个固定长度窗口。
- **适合在线分析**：预测结果可以随着查询过程持续产生，不需要等待全部数据加载完成。

在实际实现中，窗口可以看作一个先进先出的缓冲区。新数据进入窗口后，如果窗口长度超过设定值，就移除最旧的数据点。这样可以保证每一次回归都只基于最近一段数据。

### 3.3 效率优化思路

为了使算子适合时序数据库场景，我们会重点考虑以下效率问题：

- 1. 避免全量历史扫描**  
算子只维护当前窗口，避免在每个时间点重新读取或保存完整历史数据。
- 2. 减少重复计算**  
在线性回归中，斜率和截距主要依赖若干累加统计量。通过维护窗口内的统计量，可以减少每次重新遍历窗口的开销。
- 3. 控制对象创建成本**  
对高频时间序列而言，频繁创建临时对象会增加运行开销。实现时应尽量使用简单的数据结构保存窗口内容。
- 4. 支持参数化窗口**  
不同场景对响应速度和稳定性的要求不同，因此窗口长度不应写死，而应通过参数控制。
- 5. 数据库内完成计算**  
将趋势预测放在 IoTDB 查询阶段完成，可以减少数据导出量，也方便与筛选、聚合等查询操作结合使用。

### 3.4 边界情况处理

为了保证算子在实际查询中稳定运行，需要考虑以下情况：

- 窗口内有效点数不足时，暂不输出或按最小有效点数策略处理；

- 输入序列存在空值时，跳过空值或只使用有效点参与拟合；
- 参数不合法时，给出明确错误提示；
- 当窗口内横坐标没有变化、回归公式分母为 0 时，避免除零错误；
- 对非等间隔时间序列，预留按真实时间间隔建模的扩展空间。

## 3.5 系统扩展方向

在完成基础预测功能后，我们还可以从以下方向扩展：

- 输出局部斜率，用于趋势强度判断；
- 根据斜率阈值输出“上升、下降、平稳”等趋势标签；
- 增加简单异常点过滤，降低极端值对回归结果的影响；
- 支持真实时间间隔建模，使算子适应非等间隔采样数据；
- 在实验中比较直接计算和增量统计计算的性能差异。

## 3.6 数据处理流程

从系统流程上看，该 UDF 可以分为“输入读取、窗口更新、统计计算、结果输出”四个阶段。

1. **输入读取阶段：**IoTDB 查询引擎按时间顺序将数据传入 UDF，算子读取当前时间戳和对应数值。
2. **窗口更新阶段：**新数据进入窗口。如果窗口长度超过设定值，则移除最早的数据点，使窗口始终代表最近一段历史。
3. **统计计算阶段：**基于窗口内数据计算线性回归所需的统计量，并得到斜率、截距和预测值。
4. **结果输出阶段：**将预测值或趋势信息返回给查询结果，使用户可以继续 IoTDB 中观察、筛选或展示。

该流程的特点是和 IoTDB 的按行处理机制比较一致，不需要额外启动外部服务，也不需要将数据整体导出到其他分析平台。对于课程作业而言，这能较好体现 UDF 算子和时序数据库结合的价值。

# 四、预期实验设计

## 4.1 正确性实验

构造已知线性序列，例如：

$$y = 2x + 3 \quad (11)$$

在该序列上运行算子，检查预测值是否接近理论结果。该实验主要用于验证回归公式、窗口维护和预测逻辑是否正确。

## 4.2 趋势识别实验

分别构造持续上升、持续下降和基本平稳三类序列，观察输出斜率或趋势信息是否符合预期。该实验用于验证算子能否反映局部趋势方向。

## 4.3 抗噪声实验

在线性趋势上加入小幅随机扰动，观察预测结果是否仍能保持大致趋势。同时比较不同窗口长度下的输出变化，分析窗口长度对平滑性和响应速度的影响。

## 4.4 IoTDB 集成实验

将样例数据写入 IoTDB，注册 UDF 后在查询语句中调用，验证算子是否能够在 IoTDB 环境中正常运行。重点检查参数配置、输出类型、空值处理和多组数据下的稳定性。

## 4.5 性能实验

在不同数据量和不同窗口长度下统计查询耗时，观察窗口大小对运行效率的影响。如果实现增量统计优化，还可以比较优化前后的运行时间，验证该思路是否能减少重复计算。

## 4.6 评价指标

为了让实验结果更清晰，我们计划从以下几个角度评价算子效果：

| 指标      | 含义                 | 用途             |
|---------|--------------------|----------------|
| 平均绝对误差  | 预测值与真实值差值绝对值的平均值   | 衡量预测偏差大小       |
| 均方误差    | 预测误差平方的平均值         | 对较大误差更敏感       |
| 趋势方向准确率 | 预测趋势方向与真实变化方向一致的比例 | 衡量上升、下降、平稳判断效果 |
| 查询耗时    | UDF 查询执行时间         | 衡量不同窗口长度下的性能   |
| 内存占用趋势  | 窗口长度变化对内存使用的影响     | 验证固定窗口设计是否可控   |

其中，预测误差用于评价数值预测效果，趋势方向准确率用于评价趋势判断能力，查询耗时和内存占用则用于说明系统实现方面的效率。

## 4.7 预期结果分析

预期在理想线性序列上，算子的预测值应与理论结果基本一致；在带有轻微噪声的序列上，预测值可能存在一定误差，但局部趋势方向应保持稳定。随着窗口长度增加，预测曲线会更平滑，但对突发变化的响应会变慢；随着窗口长度减小，算子会更敏感，但结果也更容易受到噪声影响。

性能方面，直接遍历窗口的实现会随着窗口长度增加而增加计算开销；如果采用增量统计优化，则单点更新成本可以明显降低。通过这些实验，可以说明参数选择和效率设计之间的关系。

# 五、可行性与预期成果

## 5.1 可行性分析

该算法原理清晰，实现难度可控，适合在 Java 中封装为 IoTDB UDF。线性回归公式可以直接实现，不需要复杂外部依赖。算子只保存固定长度窗口，内存占用也比较容易控制。

我们的主要工作并不在于提出新的预测模型，而在于完成一个适合 IoTDB 查询过程的 UDF 算子，包括参数设计、窗口维护、边界处理、结果输出和实验验证。因此，该选题与“UDF 函数库算子实现”的作业要求比较匹配。

## 5.2 预期成果

我们计划完成以下成果：

- 一个可在 IoTDB 中注册和调用的短期趋势预测 UDF；
- 一套支持窗口长度、预测步长、输出模式等参数的算子设计；
- 一份使用说明，说明算子的功能、参数含义和使用方式；
- 一组实验样例，展示不同趋势序列下的预测效果；
- 若时间允许，加入增量统计优化或趋势标签输出功能。

## 5.3 实施计划

我们可以按以下步骤推进：

| 阶段   | 主要任务                   | 预期结果           |
|------|------------------------|----------------|
| 第一阶段 | 理解 IoTDB UDF 接口和参考项目结构 | 明确输入、输出和参数处理方式 |
| 第二阶段 | 完成滑动窗口和线性回归基础逻辑        | 得到可运行的基础预测算子   |
| 第三阶段 | 补充边界处理和参数校验            | 提高算子稳定性        |
| 第四阶段 | 构造实验数据并完成正确性、趋势和性能实验   | 获得可展示的实验结果     |
| 第五阶段 | 整理说明文档和展示材料            | 完成大作业交付内容      |

这样的安排可以先保证基础功能完成，再根据剩余时间考虑增量统计优化、趋势标签输出等扩展内容。

## 5.4 可能风险与应对

| 风险      | 影响           | 应对方式                      |
|---------|--------------|---------------------------|
| 算法本身较基础 | 选题的算法创新性不强   | 强调数据库内 UDF 封装、流式处理和效率优化设计 |
| 窗口过小    | 结果容易受噪声影响    | 在实验中比较不同窗口长度，并给出参数建议      |
| 窗口过大    | 对趋势变化反应较慢    | 说明窗口大小与响应速度之间的取舍          |
| 非等间隔采样  | 按点序号建模可能不够准确 | 预留按真实时间间隔建模的扩展方案          |
| 异常点干扰   | 回归结果可能偏移     | 在实验中说明局限，后续考虑加入异常点过滤      |

## 六、总结

我们计划实现一个基于滑动窗口线性回归的 IoTDB 短期趋势预测 UDF。其核心思想是在最近一段数据窗口内拟合局部直线，并利用该直线输出短期预测值或趋势信息。

虽然滑动窗口线性回归属于基础算法，但它在 IoTDB 场景中具有较好的工程适配性：计算过程简单，内存占用可控，结果容易解释，并且可以直接嵌入数据库查询流程。通过这一实现，可以完成一个从算法设计、UDF 封装、效率优化到实验验证的完整实现过程。



